

COMP9417 - Machine Learning

Homework 1: Regularized Optimization & Gradient Methods

Introduction In this homework we will explore *gradient* based optimization. Gradient based algorithms have been crucial to the development of machine learning in the last few decades. The most famous example is the backpropagation algorithm used in deep learning, which is in fact just a particular application of a simple algorithm known as (stochastic) gradient descent. We will first implement gradient descent from scratch on a deterministic problem (no data), and then extend our implementation to solve a real world regression problem.

Points Allocation There are a total of 30 marks.

- Question 1 a): 2 marks
- Question 1 b): 4 marks
- Question 1 c): 2 marks
- Question 1 d): 2 marks
- Question 1 e): 6 marks
- Question 1 f): 6 marks
- Question 1 g): 4 marks
- Question 1 h): 2 marks
- Question 1 i): 2 marks

What to Submit

- A **single PDF** file which contains solutions to each question. For each question, provide your solution in the form of text and requested plots. For some questions you will be requested to provide screen shots of code used to generate your answer — only include these when they are explicitly asked for.
- **.py file(s) containing all code you used for the project, which should be provided in a separate .zip file.** This code must match the code provided in the report.
- You may be deducted points for not following these instructions.
- You may be deducted points for poorly presented/formatted work. Please be neat and make your solutions clear. Start each question on a new page if necessary.

- You **cannot** submit a Jupyter notebook; this will receive a mark of zero. This does not stop you from developing your code in a notebook and then copying it into a .py file though, or using a tool such as **nbconvert** or similar.
- We will set up a Moodle forum for questions about this homework. Please read the existing questions before posting new questions. Please do some basic research online before posting questions. Please only post clarification questions. Any questions deemed to be *fishing* for answers will be ignored and/or deleted.
- Please check Moodle announcements for updates to this spec. It is your responsibility to check for announcements about the spec.
- Please complete your homework on your own, do not discuss your solution with other people in the course. General discussion of the problems is fine, but you must write out your own solution and acknowledge if you discussed any of the problems in your submission (including their name(s) and zID).
- As usual, we monitor all online forums such as Chegg, StackExchange, etc. Posting homework questions on these site is equivalent to plagiarism and will result in a case of academic misconduct.
- You may **not** use SymPy or any other symbolic programming toolkits to answer the derivation questions. This will result in an automatic grade of zero for the relevant question. You must do the derivations manually.

When and Where to Submit

- **Due date: Monday June 22, 2026 by 5pm.** Please note that the forum will not be actively monitored on weekends.
- Late submissions will incur a penalty of 5% per day **from the maximum achievable grade**. For example, if you achieve a grade of 80/100 but you submitted 3 days late, then your final grade will be $80 - 3 \times 5 = 65$. Submissions that are more than 5 days late will receive a mark of zero.
- Submission must be made on **Moodle**, **no exceptions**.

Question 1. Gradient Based Optimization

The general framework for a gradient method for finding a minimizer of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is defined by

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x_k), \quad k = 0, 1, 2, \dots, \quad (1)$$

where $\alpha_k > 0$ is known as the step size, or learning rate. Consider the following simple example of minimizing the function $g(x) = 2\sqrt{x^3 + 1}$. We first note that $g'(x) = 3x^2(x^3 + 1)^{-1/2}$. We then need to choose a starting value of x , say $x^{(0)} = 1$. Let's also take the step size to be constant, $\alpha_k = \alpha = 0.1$. Then we have the following iterations:

$$x^{(1)} = x^{(0)} - 0.1 \times 3(x^{(0)})^2((x^{(0)})^3 + 1)^{-1/2} = 0.7878679656440357$$

$$x^{(2)} = x^{(1)} - 0.1 \times 3(x^{(1)})^2((x^{(1)})^3 + 1)^{-1/2} = 0.6352617090300827$$

$$x^{(3)} = 0.5272505146487477$$

$\vdots$

and this continues until we terminate the algorithm (as a quick exercise for your own benefit, code this up and compare it to the true minimum of the function which is $x_* = -1$ ¹). This idea works for functions that have vector valued inputs, which is often the case in machine learning. For example, when we minimize a loss function we do so with respect to a weight vector, β . When we take the step-size to be constant at each iteration, this algorithm is known as gradient descent. For the entirety of this question, **do not use any existing implementations of gradient methods, doing so will result in an automatic mark of zero for the entire question.**

(a) Consider the following optimisation problem:

$$\min_{x \in \mathbb{R}^n} f(x),$$

where

$$f(x) = \frac{1}{2} \|Ax - b\|_2^2 + \frac{\gamma}{2} \|x\|_2^2,$$

and where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ are defined as

$$A = \begin{bmatrix} 3 & 2 & 0 & -1 \\ -1 & 3 & 0 & 2 \\ 0 & -4 & -2 & 7 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 1 \\ -4 \end{bmatrix},$$

and γ is a positive constant. Run gradient descent on f using a step size of $\alpha = 0.01$ and $\gamma = 2$ and starting point of $x^{(0)} = (1, 1, 1, 1)$. You will need to terminate the algorithm when the following condition is met: $\|\nabla f(x^{(k)})\|_2 < 0.001$. In your answer, clearly write down the version of the gradient steps (1) for this problem. Also, print out the first 5 and last 5 values of $x^{(k)}$, clearly indicating the value of k , in the form:

$$\begin{aligned} k = 0, & \quad x^{(k)} = [1, 1, 1, 1] \\ k = 1, & \quad x^{(k)} = \dots \\ k = 2, & \quad x^{(k)} = \dots \\ & \quad \vdots \end{aligned}$$

¹Does the algorithm converge to the true minimizer? Why/Why not?

What to submit: an equation outlining the explicit gradient update, a print out of the first 5 ($k = 5$ inclusive) and last 5 rows of your iterations. Use the round function to round your numbers to 4 decimal places. Include a screen shot of any code used for this section and a copy of your python code in solutions.py.

Consider now a slightly different problem: let $y, \beta \in \mathbb{R}^p$ and $\lambda > 0$. Further, we define the matrix $W \in \mathbb{R}^{(p-2) \times p}$ as

$$W = \begin{bmatrix} 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & 1 & -2 & 1 \\ & & & & 1 & -2 & 1 \end{bmatrix},$$

where blanks denote zero elements.² Define the loss function:

$$L(\beta) = \frac{1}{2p} \|y - \beta\|_2^2 + \lambda \|W\beta\|_2^2. \quad (2)$$

The following code allows you to load in the data needed for this problem³:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 t_var = np.load("t_var.npy")
4 y_var = np.load("y_var.npy")
5 plt.plot(t_var, y_var)
6 plt.show()
7
```

Note, the t variable is purely for plotting purposes, it should not appear in any of your calculations.

(b) Show that

$$\hat{\beta} = \arg \min_{\beta} L(\beta) = (I + 2\lambda p W^T W)^{-1} y.$$

Update the following code⁴ so that it returns a plot of $\hat{\beta}$ and calculates $L(\hat{\beta})$. Only in your code implementation, set $\lambda = 0.9$.

```
1 def create_W(p):
2     ## generate W which is a p-2 x p matrix as defined in the question
3     W = np.zeros((p-2, p))
4     b = np.array([1, -2, 1])
5     for i in range(p-2):
6         W[i, i:i+3] = b
7     return W
8
9 def loss(beta, y, W, L):
10    ## compute loss for a given vector beta for data y, matrix W, regularization
11    ## parameter L (lambda)
12    # your code here
```

²If it is not already clear: for the first row of W : $W_{11} = 1, W_{12} = -2, W_{13} = 1$ and $W_{1j} = 0$ for any $j \geq 4$. For the second row of W : $W_{21} = 0, W_{22} = 1, W_{23} = -2, W_{24} = 1$ and $W_{2j} = 0$ for any $j \geq 5$ and so on.

³a copy of this code is provided in code_student.py

⁴a copy of this code is provided in code_student.py

```

12     return loss_val
13
14 ## your code here, e.g. compute betahat and loss, and set other params..
15
16 plt.plot(t_var, y_var, zorder=1, color='red', label='truth')
17 plt.plot(t_var, beta_hat, zorder=3, color='blue',
18          linewidth=2, linestyle='--', label='fit')
19 plt.legend(loc='best')
20 plt.title(f"L(beta_hat) = {loss(beta_hat, y, W, L)}")
21 plt.show()
22

```

What to submit: a closed form expression along with your working, a single plot and a screen shot of your code along with a copy of your code in your .py file.

- (c) Write out each of the two terms that make up the loss function ($\frac{1}{2p}\|y - \beta\|_2^2$ and $\lambda\|W\beta\|_2^2$) explicitly using summations. Use this representation to explain the role played by each of the two terms. Be as specific as possible. *What to submit: your answer, and any working either typed or handwritten.*
- (d) Show that we can write (2) in the following way:

$$L(\beta) = \frac{1}{p} \sum_{j=1}^p L_j(\beta),$$

where $L_j(\beta)$ depends on the data $y_1, \dots, y_p$ only through y_j . Further, show that

$$\nabla L_j(\beta) = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} - (y_j - \beta_j) \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} + 2\lambda W^T W \beta, \quad j = 1, \dots, p.$$

Note that the first vector is the p -dimensional vector with zero everywhere except for the j -th index. Take a look at the supplementary material if you are confused by the notation. *What to submit: your answer, and any working either typed or handwritten.*

- (e) In this question, you will implement (batch) GD from scratch to minimize the loss function (2). Use an initial estimate $\beta^{(0)} = \mathbf{1}_p$ (the p -dimensional vector of ones), and $\lambda = 0.001$ and run the algorithm for 1000 epochs (an epoch is one pass over the entire data, so a single GD step). Repeat this for the following step sizes:

$$\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.3, 0.6, 1.2, 2\}$$

To monitor the performance of the algorithm, we will plot the value

$$\Delta^{(k)} = L(\beta^{(k)}) - L(\hat{\beta}),$$

where $\hat{\beta}$ is the true (closed form) solution derived earlier (but with $\lambda = 0.001$ now for consistency). Present your results in a single 3×3 grid plot, with each subplot showing the progression of $\Delta^{(k)}$ when running GD with a specific step-size. State which step-size you think is best in terms of speed of convergence. *What to submit: a single plot. Include a screen shot of any code used for this section and a copy of your python code in solutions.py.*

- (f) We will now implement SGD from scratch to solve (2). Use an initial estimate $\beta^{(0)} = 1_p$ (the vector of ones) and $\lambda = 0.001$ and run the algorithm for 4 epochs (this means a total of $4p$ updates of β). Repeat this for the following step sizes:

$$\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.3, 0.6, 1.2, 2\}$$

Present an analogous single 3×3 grid plot as in the previous question. Instead of choosing an index randomly at each step of SGD, we will cycle through the observations in the order they are stored in y to ensure consistent results. Report the best step-size choice. In some cases you might observe that the value of $\Delta^{(k)}$ jumps up and down, and this is not something you would have seen using batch GD. Why do you think this might be happening?

What to submit: a single plot and some commentary. Include a screen shot of any code used for this section and a copy of your python code in solutions.py.

An alternative Coordinate Based scheme: In GD, SGD and mini-batch GD, we always update the entire p -dimensional vector β at each iteration. An alternative approach is to update each of the p parameters individually. To make this idea more clear, we write the loss function of interest $L(\beta)$ as $L(\beta_1, \beta_2, \dots, \beta_p)$. We initialize $\beta^{(0)}$, and then solve for $k = 1, 2, 3, \dots$,

$$\begin{aligned}\beta_1^{(k)} &= \arg \min_{\beta_1} L(\beta_1, \beta_2^{(k-1)}, \beta_3^{(k-1)}, \dots, \beta_p^{(k-1)}) \\ \beta_2^{(k)} &= \arg \min_{\beta_2} L(\beta_1^{(k)}, \beta_2, \beta_3^{(k-1)}, \dots, \beta_p^{(k-1)}) \\ &\vdots \\ \beta_p^{(k)} &= \arg \min_{\beta_p} L(\beta_1^{(k)}, \beta_2^{(k)}, \beta_3^{(k)}, \dots, \beta_p).\end{aligned}$$

Note that each of the minimizations is over a single (1-dimensional) coordinate of β , and also that as soon as we update $\beta_j^{(k)}$, we use the new value when solving the update for $\beta_{j+1}^{(k)}$ and so on. The idea is then to cycle through these coordinate level updates until convergence. In the next two parts we will implement this algorithm from scratch for the problem we have been working on (2).

- (g) Derive closed-form expressions for $\hat{\beta}_1, \hat{\beta}_2, \dots, \hat{\beta}_p$ where for $j = 1, 2, \dots, p$:

$$\hat{\beta}_j = \arg \min_{\beta_j} L(\beta_1, \dots, \beta_{j-1}, \beta_j, \beta_{j+1}, \dots, \beta_p).$$

What to submit: a closed form expression along with your working.

Hint: Be careful, this is not as straight-forward as it might seem at first. It is recommended to choose a value for p , e.g. $p = 8$ and first write out the expression in terms of summations. Then take derivatives to get the closed form expressions.

- (h) Implement both gradient descent and the coordinate scheme in code (from scratch) and apply it to the provided data. In your implementation:
- Use $\lambda = 0.001$ for the coordinate scheme, and step-size $\alpha = 1$ for your gradient descent scheme.
 - Initialize both algorithms with $\beta = 1_p$, the p -dimensional vector of ones.
 - For the coordinate scheme, be sure to update the β_j 's in order (i.e. 1,2,3,...)

- For your coordinate scheme, terminate the algorithm after 1000 updates (each time you update a single coordinate, that counts as an update.)
- For your GD scheme, terminate the algorithm after 1000 epochs.
- Create a single plot of k vs $\Delta^{(k)} = L(\beta^{(k)}) - L(\hat{\beta})$, where $\hat{\beta}$ is the closed form expression derived earlier. Your plot should have both the coordinate scheme (blue) and GD (green) displayed and should start from $k = 0$. Your plot should have a legend.

What to submit: a single plot and a screen shot of your code along with a copy of your code in your .py file.

- (i) Based on your answer to the previous part, when would you prefer GD? When would you prefer the coordinate scheme? *What to submit: Some commentary.*

Supplementary: Background on Gradient Descent As noted in the lectures, there are a few variants of gradient descent that we will briefly outline here. Recall that in gradient descent our update rule is

$$\beta^{(k+1)} = \beta^{(k)} - \alpha_k \nabla L(\beta^{(k)}), \quad k = 0, 1, 2, \dots,$$

where $L(\beta)$ is the loss function that we are trying to minimize. In machine learning, it is often the case that the loss function takes the form

$$L(\beta) = \frac{1}{n} \sum_{i=1}^n L_i(\beta),$$

i.e. the loss is an average of n functions that we have labelled L_i , and each L_i depends on the data only through (x_i, y_i) . It then follows that the gradient is also an average of the form

$$\nabla L(\beta) = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\beta).$$

We can now define some popular variants of gradient descent .

- (i) Gradient Descent (GD) (also referred to as batch gradient descent): here we use the full gradient, as in we take the average over all n terms, so our update rule is:

$$\beta^{(k+1)} = \beta^{(k)} - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla L_i(\beta^{(k)}), \quad k = 0, 1, 2, \dots$$

- (ii) Stochastic Gradient Descent (SGD): instead of considering all n terms, at the k -th step we choose an index i_k randomly from $\{1, \dots, n\}$, and update

$$\beta^{(k+1)} = \beta^{(k)} - \alpha_k \nabla L_{i_k}(\beta^{(k)}), \quad k = 0, 1, 2, \dots$$

Here, we are approximating the full gradient $\nabla L(\beta)$ using $\nabla L_{i_k}(\beta)$.

- (iii) Mini-Batch Gradient Descent: GD (using all terms) and SGD (using a single term) represents the two possible extremes. In mini-batch GD we choose batches of size $1 < B < n$ randomly at each step, call their indices $\{i_{k_1}, i_{k_2}, \dots, i_{k_B}\}$, and then we update

$$\beta^{(k+1)} = \beta^{(k)} - \frac{\alpha_k}{B} \sum_{j=1}^B \nabla L_{i_j}(\beta^{(k)}), \quad k = 0, 1, 2, \dots,$$

so we are still approximating the full gradient but using more than a single element as is done in SGD.